

Best AI Projects to Build in 2026 to Make Your Resume Unbeatable.

For engineers who ship real things

01 Multi-Agent

02 RAG + Eval

03 Local LLM

04 Code Review CLI

05 Multimodal

2026 Edition · Not a Tutorial · Real Depth

THE UNCOMFORTABLE TRUTH

Everyone is building **the same thing.**

OVERUSED

Basic Chatbot

Flask + OpenAI API + React.
Recruiters have seen 40 of these
before lunch today.

OVERUSED

PDF Q&A App

Upload → chunk → embed → retrieve.
Tutorial-tier.
No differentiator.

OVERUSED

Summariser App

Paste article → get summary.
This is a single API call.
Not an engineering project.

Having an AI project on your resume is no longer a differentiator. It's table stakes.
What separates candidates now is technical depth, evaluation, and real-world judgment.

WHAT HIRING MANAGERS ACTUALLY CHECK

3 Questions Every Recruiter Asks

1 Did they actually build this, or follow a tutorial?

Signs they look for: commit history, feature branches, issues closed, iterations, bugs fixed. Clean 1-commit repos are a red flag.

2 Do they understand the architecture & tradeoffs?

Not just 'it uses GPT-4.' Why this chunking strategy? Why this vector store? What were the latency constraints?

3 Did they care enough to document it properly?

A great README is a proxy for how you communicate on a team. Architecture diagram + numbers + decisions = a strong signal.

Build projects like you're shipping them to real users.
Because the best ones? You actually can.

01

ADVANCED

Multi-Agent Orchestration System

LangGraph · GitHub API · Docker · Python

What You're Building

Design a system of specialized agents, each owning a specific task, coordinated by an orchestrator that routes work based on live state and conditional logic.

Concrete idea: A system that takes a GitHub Issue, researches the codebase, drafts a fix, writes tests, and opens a pull request — autonomously.

LangGraph

GitHub API

Docker Sandbox

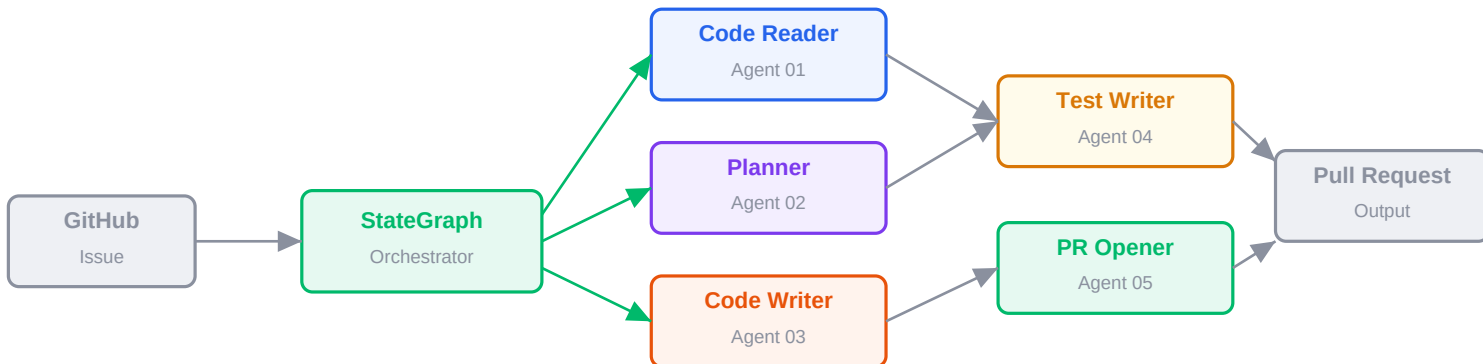
Python

Why This Hits in Interviews

- Shows stateful reasoning
 - understand graph-based agents vs simple chains
- Real engineering problems
 - partial failures, retries, tool call errors
- Mirrors what AI labs build
 - Cognition, Cursor, every serious AI company
- Conversation goldmine
 - 15+ mins of deep technical interview material

PROJECT 01 — ARCHITECTURE

LangGraph StateGraph: Multi-Agent Flow



```
AgentState { issue, code_context, plan, patch, tests, pr_url }
```

```
langgraph_workflow.py
```

```
# LangGraph: each agent is a node, edges define control flow
from langgraph.graph import StateGraph, END

workflow = StateGraph(AgentState)
workflow.add_node("code_reader", code_reader_agent)
workflow.add_conditional_edges("planner", route_by_complexity,
    {"simple": "code_writer", "complex": "research_agent"})
```

Production RAG Pipeline + Evaluation Layer

LlamaIndex · RAGAS · Qdrant · Cohere Rerank · BM25

Build a retrieval-augmented generation system over a complex domain — then build the evaluation harness to

The money line:
'Context precision: 0.61 (naive RAG)
→ 0.84 (semantic chunking).'

That's an engineering result.

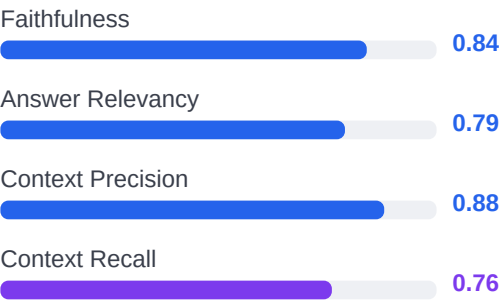
LlamaIndex

RAGAS

Qdrant

BM25

RAGAS Evaluation Metrics



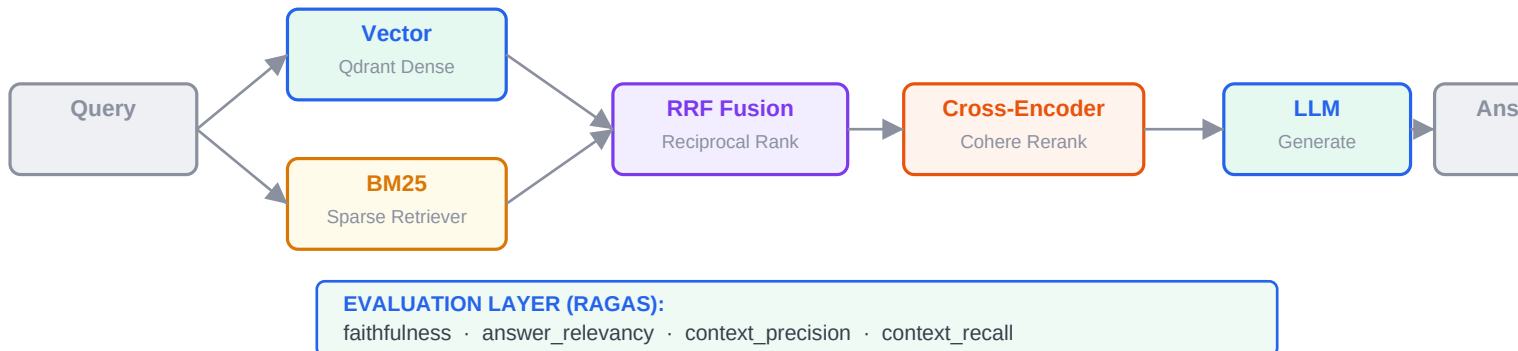
Before (naive chunking): context_precision = 0.61
After (semantic chunking): context_precision = 0.84 ↑ +38%

Why most engineers miss this:

Most build the RAG app. Very few benchmark it. RAGAS metrics = tutorial → engineering artifact.

PROJECT 02 — ARCHITECTURE

Advanced RAG: Dense + Sparse + Reranker



ragas_eval.py

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision

result = evaluate(dataset=test_set,
                  metrics=[faithfulness, answer_relevancy, context_precision])
# result['context_precision']: 0.61 → 0.84 ← PUT THIS IN YOUR README
```

Naive RAG is a single prompt. Production RAG is a measurable system.
The evaluation layer is what makes this project interview-worthy.

03

UNDERBUILT = YOUR EDGE

Local LLM Deployment on Edge Hardware

Ollama · llama.cpp · GGUF format · NVIDIA Jetson · Raspberry Pi

Deploy a quantized LLM to run inference locally — no cloud API. Healthcare, defense, and finance can't send sensitive data to OpenAI servers. You solve that.

Most junior engineers can't do this.
The ability to deploy capable models locally
is genuinely commercially valuable.
Be the person who can.

Ollama

llama.cpp

GGUF format

```
local inference — llama.cpp

$ ollama run llama3.2:3b-q4_K_M
pulling manifest... ✓
loading model (2.1 GB)...
model loaded in 1.2s

>>> Summarise this patient note:
Patient presents with chronic lower back...

✓ 14.3 tok/s · no API call · zero data egress
```


THE ENGINEERING ARTIFACT — PUT THIS IN YOUR README

Quantization Benchmark: Llama 3.2 3B (MacBook M3)

Model / Quant	File Size	RAM	Tok/sec	Quality (MMLU)	Best For
llama3.2:3b-fp16	6.4 GB	7.1 GB	8.2	Baseline	Max quality
llama3.2:3b-Q8_0	3.4 GB	4.0 GB	11.4	~99% FP16	Quality / size tradeoff
llama3.2:3b-Q4_K_M ★	2.0 GB	2.5 GB	18.7	~95% FP16	Recommended for edge
llama3.2:3b-Q3_K_S	1.5 GB	1.9 GB	24.1	~89% FP16	Raspberry Pi / low RAM

```
local_inference.py

from llama_cpp import Llama

llm = Llama(model_path="./llama-3.2-3b-Q4_K_M.gguf",
             n_ctx=4096, n_gpu_layers=-1) # offload to GPU/Metal
output = llm("Summarise this patient report: " + doc, max_tokens=512)
```

The benchmark table IS the project. Measure, document, and present tradeoffs. This is what engineers at real companies do — not just 'it works on my machine.'

04

BUILD FOR DEVS

AI-Powered Code Review CLI Tool

Python CLI · GitHub Actions · Pydantic · Structured Output

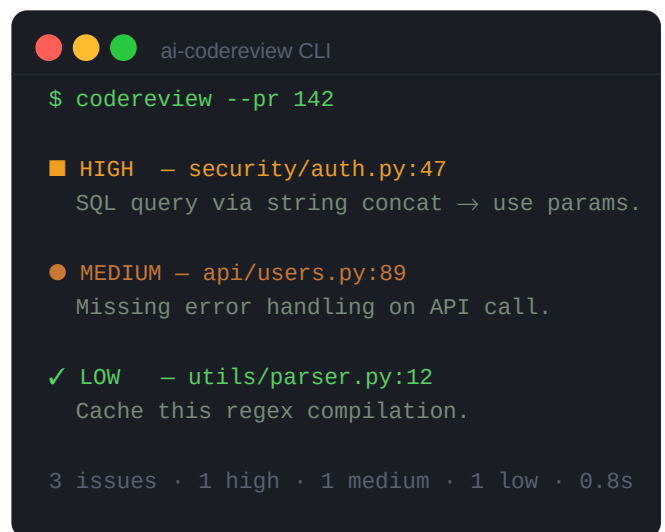
Build something other developers would actually use. The user is a developer — the most discerning early adopter on the planet. Impress them.

Add a GitHub Actions integration and it automatically runs on every PR. Now it's infrastructure, not just a script.

Python CLI

GitHub Actions

Pydantic



```
ai-codereview CLI

$ codereview --pr 142

■ HIGH — security/auth.py:47
  SQL query via string concat → use params.

● MEDIUM — api/users.py:89
  Missing error handling on API call.

✓ LOW — utils/parser.py:12
  Cache this regex compilation.

3 issues · 1 high · 1 medium · 1 low · 0.8s
```

PROJECT 04 — THE INTERESTING ENGINEERING PROBLEMS

Structured Output + Schema-Enforced Reviews

```
code_review_schema.py

from pydantic import BaseModel
from enum import Enum

class Severity(str, Enum):
    HIGH = "high" | MEDIUM = "medium" | LOW = "low"

class ReviewIssue(BaseModel):
    file: str | line_number: int | severity: Severity
    category: str # "security" | "performance" | "test_coverage"
    explanation: str | suggested_fix: str

# Force LLM to return this schema — no regex hacks on free text
result = client.beta.chat.completions.parse(
    model="gpt-4o", response_format=CodeReviewResult)
```

Challenge 1

Large diffs > context window

Smart chunking that preserves function boundaries
— not naive line splits.

Challenge 2

Reliable structured output

Schema-enforced via Pydantic. Zero regex hacking
on free-form LLM text.

Challenge 3

Evaluation quality

Test set of known-bad code with expected feedback
categories. Measurable.

The secret weapon: structured output means your reviews are parseable, testable, and directly usable in CI pipelines. This is production thinking.

05

REAL USERS REQUIRED

Multimodal AI App with Real Users

GPT-4o Vision · Next.js · Claude 3.5 Sonnet · Vision Classifier

This one has a deliberate extra requirement: real users. Not 'I showed it to my friends.' Actual strangers using it. Ship it.

THE RESUME LINE THAT MATTERS
"Served 1,200 users · 340 feedback items · iterated 3x on claude 3.5 sonnet"

Shipping to real users is the only thing that separates a project from a product in an interviewer's eyes.

GPT-4o Vision

Next.js

Vercel

Concrete Idea: Visual Debugging Assistant

User uploads a screenshot of an error: terminal output, UI bug, broken chart.

System returns a structured diagnosis and a suggested fix.

Key Engineering Layer: Vision Classifier → Router

Don't pass the raw image to the LLM. First classify: terminal error / UI bug / chart. Route each type to a specialized prompt chain.

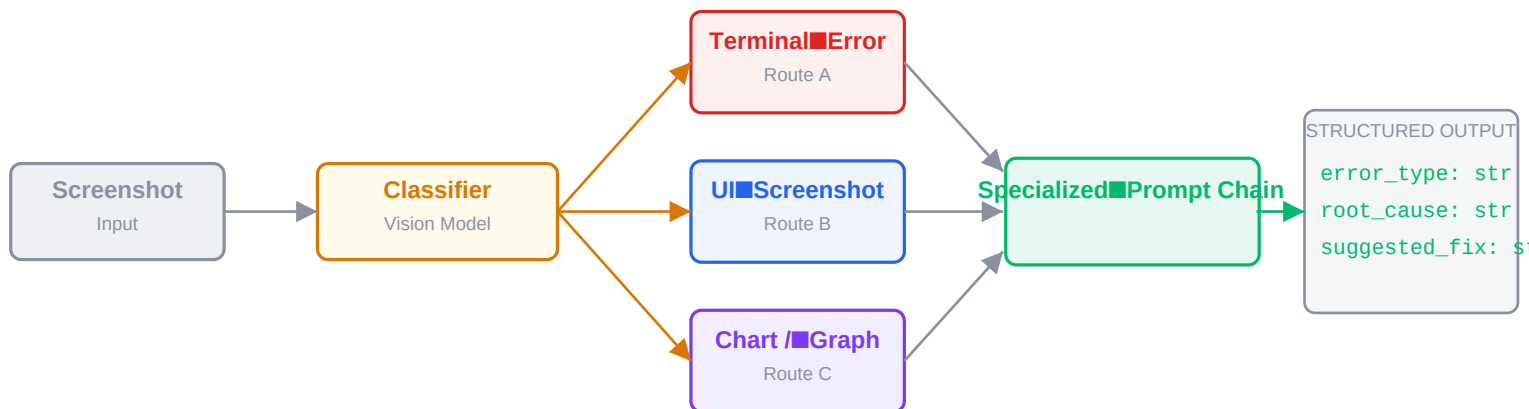
This is what turns a demo into a product.

SHIP ON:

Product Hunt · r/webdev · HN Show HN

PROJECT 05 — MULTIMODAL PIPELINE

Screenshot → Classify → Route → Structured Output



This is what separates projects from products.

When you say: 'I shipped this, real people used it, here's what broke, here's how I fixed it'

— that is an incredibly powerful signal. It shows you can close the loop from idea to production.

vision_classifier.py

```
# Classify first, then route to specialized prompt chain
response = openai.chat.completions.create(
    model="gpt-4o", messages=[{"role": "user", "content": [
        {"type": "image_url", "image_url": {"url": image_url}},
        {"type": "text", "text": "Classify: terminal_error|ui_bug|chart"}
    ]}], response_format=ClassificationResult)
```

THE COMPLETE LIST

5 Projects. Real Depth. Real Jobs.

#	Project	Core Tech	Difficulty	Interview Signal
01	Multi-Agent Orchestration System	LangGraph, GitHub API, Docker	★★★★★	Systems thinking · Agentic AI
02	Production RAG + Evaluation Layer	LlamaIndex, RAGAS, Qdrant, BM25	★★★★■	ML engineering · Metrics
03	Local LLM Deployment on Edge	Ollama, llama.cpp, GGUF	★★★★■	ML systems · Privacy-first AI
04	AI-Powered Code Review CLI	Python, Pydantic, GitHub Actions	★★★██	DX taste · Structured outputs
05	Multimodal App with Real Users	GPT-4o Vision, Next.js, Vercel	★★★★■	Product thinking · Shipping

You don't need all five. Pick ONE. Build it properly with a solid README, real benchmarks, and documented architectural decisions. One excellent project beats five mediocre ones every time.

HOW TO PRESENT THESE PROJECTS

The GitHub README Formula That Works

1 One-sentence description of what + why

"A multi-agent system that takes a GitHub issue as input and autonomously opens a PR with tests."

2 Architecture diagram (Excalidraw / Mermaid.js)

Even a rough one. A diagram is a proxy for whether you understand your own system.

3 Results / Benchmarks section with real numbers

"Context precision: 0.61 (naive) → 0.84 (semantic chunking).
Latency: 280ms p95."

4 Technical Decisions section ← the secret weapon

2–3 choices you made and why. Shows reasoning under constraints. Most people skip this.

```
README.md structure

## What It Does
One sentence. Clear. No jargon.

## Architecture


## Results


| Metric        | Before | After |
|---------------|--------|-------|
| ctx_precision | 0.61   | 0.84  |



## Technical Decisions
**Why Qdrant over Pinecone?**
Self-hosted, zero data egress,
better metadata filtering.

## Running Locally
```

HOW TO WRITE THE RESUME BULLETS

Before vs. After

✗ BEFORE — generic, forgettable

Built an AI chatbot using Python and OpenAI API.

✓ AFTER — specific, measurable, technical

Designed a multi-agent orchestration system using LangGraph to automate code review workflows; reduced average review latency by 40% in internal benchmarks across a 50-file codebase.

✗ BEFORE — generic, forgettable

Built a RAG app to answer questions about PDF documents.

✓ AFTER — specific, measurable, technical

Built a production-grade RAG pipeline with hybrid dense/sparse retrieval and cross-encoder reranking; evaluated with RAGAS — improved context precision from 0.61 to 0.84 via semantic chunking.

✗ BEFORE — generic, forgettable

Deployed a local LLM on my laptop using Ollama.

✓ AFTER — specific, measurable, technical

Deployed quantized Llama 3.2 (Q4_K_M, 2.1 GB) on edge hardware for privacy-constrained healthcare document summarisation; benchmarked tradeoffs across 4 quantisation levels at 18.7 tok/s.

THE ONE THING TO TAKE AWAY

Don't plan it for three weeks.
Open your editor.
Start a new repo.
Write the first file.

The engineers getting the best jobs aren't the smartest in the room. They're the ones who can look at a real problem, build something, and explain what they built clearly.

01 Multi-Agent Orchestration

02 Production RAG + Eval

03 Local LLM Deployment

04 Code Review CLI

05 Multimodal + Real Users

Pick ONE. Build the actual thing.

